
INTERNET & WEB TECHNOLOGIES

Dr. Eric HITIMANA

Contents

~~1. INTRODUCTION TO INTERNET AND WEB~~

~~2. HTML~~

~~3. CSS~~

4. JAVASCRIPT (JS)

JavaScript - Contents

- 4.1 Introduction
 - 4.2 How to start
 - 4.3 Where to put
 - 4.4 Statement
 - 4.5 Variables
 - 4.6 Operators
 - 4.7 Conditional Statements
 - 4.8 Popup Boxes
 - 4. 9 Functions
 - 4. 10 LOOPS
 - 4. 11 Breaks, and Continue
 - 4. 12 For...in statement
 - 4. 13 Events
 - 4. 14 Objects
 - 4. 15 Form Validation
 - 4. 16 JSON
 - 4.17 Data Fetching with AJAX
-

4.1 Introduction

- What is JavaScript?
 - **JavaScript** was designed to add interactivity to HTML pages.
 - JavaScript is originally a scripting language, but it is now a full-featured programming language.
 - A scripting language is a lightweight programming language
 - JavaScript is an interpreted language (executed without preliminary compilation)
 - It is the most popular language on the internet, and works in all major browsers
-

4.1 Introduction (...)

- Are Java and JavaScript the same?
 - **No**
 - Java and JavaScript are two completely different languages in both **concept and design**
 - Java developed with Sun Microsystems is a powerful and much more **complex programming language** – in the same category as C and C++
 - **JavaScript** was developed primarily for web development and is used to create interactive web pages and web applications. Today, it can also run on servers through environments such as **Node.js**.
-

4.1 Introduction (...)

- What can JavaScript do?
 - **JavaScript gives HTML designers programming tools**
 - **JavaScript can put dynamic text into an HTML page** the statement like `document.write("<h1>"+name+"</h1>")` can write variable text into an HTML page.
 - **JavaScript can react to events:** JavaScript can be set to execute when something happens, like when a page has finished loading or when a user clicks on an HTML element
 - **JavaScript can be used to validate data:** Validating form data before it is submitted to a server
 - **JavaScript can be used for server-side programming** through Node.js.
 - JavaScript can be used to **develop mobile applications**.
 - JavaScript can be used to **build Single-Page Applications (SPAs) and Progressive Web Applications (PWAs)**.
 - **JavaScript can communicate with web servers (AJAX/Fetch).**
 - JavaScript can **automate repetitive tasks and workflows**.
 - JavaScript can **support modern Artificial Intelligence and Machine Learning applications in web environments**.
-

4.2 How to Start

- The HTML **<script>** tag is used to insert JavaScript into an HTML page.
- Using JavaScript in HTML to write text on a web page

```
<html>
<body>
  <script type="text/javascript">
    document.write("Hello World!");
  </script>
</body>
</html>
```

- You can also add HTML tags to the JavaScript

```
<html>
<body>
  <script type="text/javascript">
    document.write("<h1>Hello World!</h1>");
  </script>
</body>
</html>
```

4.2 How to Start (...)

- The general syntax for writing JavaScript

```
<html>
<body>
<script type="text/javascript">
...
</script>
</body>
</html>
```

- The **document.write** command is a standard JavaScript command for writing output to a page

4.3 Where to put JavaScript

1. JavaScript in the **body section** will be executed WHILE the pages loads

```
<html>
<head>
</head>

<body>
<script type="text/javascript">
document.write("This message is written by JavaScript");
</script>
</body>

</html>
```

4.3 Where to put JavaScript (...)

2. JavaScript in the **head section** will be executed when CALLED
 - Eg: **Scripts in <head>**

```
<html>
<head>
<script type="text/javascript">
function message()
{
    alert("This alert box was called with the onload event");
}
</script>
</head>

<body onload="message()">
</body>
</html>
```

4.3 Where to put JavaScript (...)

- **Eg: Scripts in <head> and <body>**
 - You can place an unlimited number of scripts in your document, so you can have scripts in both the body and the head section.

```
<html>
<head>
<script type="text/javascript">
....
</script>
</head>
<body>
<script type="text/javascript">
....
</script>
</body>
```

4.3 Where to put JavaScript (...)

3. Using an External JavaScript

- If you want to run the same JavaScript on several pages, without having to write the same script on every page, you can write the JavaScript in an external file.
- It needs to be saved as **.js** file extension
- The external script cannot contain the `<script>` tag

```
<html>  
<head>  
  <script type="text/javascript" src="xxx.js"></script>  
</head>  
<body>  
</body>  
</html>
```

4.4 JavaScript Statement

- JavaScript is a sequence of statements to be executed by the browser
- It is **Case Sensitive** unlike HTML which is case insensitive
- A JavaScript statement is a command to a browser. The purpose of the command is to tell the browser what to do.

```
document.write("Hello Dolly");
```

4.4 JavaScript Statement (...)

- **JavaScript code**

- JavaScript code (or just JavaScript) is a sequence of JavaScript statements.
- Each statement is executed by the browser in the sequence in which they are written.

```
<script type="text/javascript">  
document.write("<h1>This is a heading</h1>");  
document.write("<p>This is a paragraph.</p>");  
document.write("<p>This is another paragraph.</p>");  
</script>
```

4.4 JavaScript Statement (...)

- **JavaScript Blocks**

- JavaScript statements can be grouped together in blocks.
- Blocks start with a left curly bracket {, and ends with a right curly bracket }.
- The purpose of a block is to make the sequence of statements execute together.

```
<script type="text/javascript">
{
document.write("<h1>This is a heading</h1>");
document.write("<p>This is a paragraph.</p>");
document.write("<p>This is another paragraph.</p>");
}
</script>
```

4.4 JavaScript Statement (...)

- **JavaScript comments**

- JavaScript comments can be added to explain the JavaScript or to make the code more readable.

```
<script type="text/javascript">  
// Write a heading  
document.write("<h1>This is a heading</h1>");  
// Write two paragraphs:  
document.write("<p>This is a paragraph.</p>");  
document.write("<p>This is another paragraph.</p>");  
</script>
```

```
<script type="text/javascript">  
/*  
The code below will write  
one heading and two paragraphs  
*/  
document.write("<h1>This is a heading</h1>");  
document.write("<p>This is a paragraph.</p>");  
document.write("<p>This is another paragraph.</p>");  
</script>
```

4.5 JavaScript Variables

- Variables are containers for storing information
- **Declaring or creating JavaScript Variables**

```
var X;  
var carname;
```

- **Assigning values to a variable**

```
var x=5;  
var carname="Volvo";
```

- **Assigning Values to Undeclared JavaScript Variables**
 - If you assign values to variables that have not yet been declared, the variables will automatically be declared.

4.6 JavaScript Operators

- Operators are for doing some **mathematical operations**.
- Given that `y=5`;

Operator	Description	Example	Result
+	Addition	<code>x=y+2</code>	<code>x=7</code>
-	Subtraction	<code>x=y-2</code>	<code>x=3</code>
*	Multiplication	<code>x=y*2</code>	<code>x=10</code>
/	Division	<code>x=y/2</code>	<code>x=2.5</code>
%	Modulus (division remainder)	<code>x=y%2</code>	<code>x=1</code>
++	Increment	<code>x=++y</code>	<code>x=6</code>
--	Decrement	<code>x=--y</code>	<code>x=4</code>

4.6 JavaScript Operators (...)

- **JavaScript Assignment Operators**
- Given that $x=10$ and $y=5$

Operator	Example	Same As	Result
=	$x=y$		$x=5$
+=	$x+=y$	$x=x+y$	$x=15$
-=	$x-=y$	$x=x-y$	$x=5$
=	$x=y$	$x=x*y$	$x=50$
/=	$x/=y$	$x=x/y$	$x=2$
%=	$x\%=y$	$x=x\%y$	$x=0$

4.6 JavaScript Operators (...)

- **The + Operator used on Strings**

- The + operator can also be used to add string variables or text values together.
- To add two or more string variables together, use the + operator

```
txt1="What a very";  
txt2="nice day";  
txt3=txt1+txt2;
```

- To insert a space, better to add space between variables

```
txt1="What a very";  
txt2="nice day";  
txt3=txt1+" "+txt2;
```

4.6 JavaScript Operators (...)

- **JavaScript Comparison and Logical Operators**
- Comparison and logical operators are used to test for TRUE or FALSE.
- They can be used in conditional statements to compare values and take action depending on the results
- **Comparison Operators**

– Given x=5;

Operator	Description	Example
==	is equal to	x==8 is false
===	is exactly equal to (value and type)	x===5 is true x==="5" is false
!=	is not equal	x!=8 is true
>	is greater than	x>8 is false
<	is less than	x<8 is true
>=	is greater than or equal to	x>=8 is false
<=	is less than or equal to	x<=8 is true

4.6 JavaScript Operators (...)

- **Logical Operators**

- They are used to determine the logic between variables or values
- Given that **x=6** and **y=3**;

Operator	Description	Example
&&	and	(x < 10 && y > 1) is true
	or	(x==5 y==5) is false
!	not	!(x==y) is true

4.6 JavaScript Operators (...)

- **Conditional Operator**

- ✓ JavaScript also contains a conditional operator that assigns a value to a variable based on some condition.

- ✓ **Syntax:**

```
variablename=(condition)?value1:value2
```

- ✓ E.g:

```
greeting=(visitor=="PRES")?"Dear President ":"Dear ";
```

3.7 JavaScript Conditional Statements

- Conditional statements are used to perform different actions based on different conditions.
 - ✓ **if Statement:** use this statement to execute some code only if a specified condition is true
 - ✓ **if....else statement:** use this statement to execute some code if the condition is true and another code if the condition is false
 - ✓ **if-...else if...else statement:** use this statement to select one of many blocks of code to be executed
 - ✓ **switch statement:** use this statement to select one of many blocks of code to be executed
-

3.7 JavaScript IF—ELSE Statements

- If statement.

```
if(condition)
{
  code to be executed if condition is true
}
```

- Example

```
var time= 10

if (time<10)
{
  document.write("<b>Good morning</b>");
}
</script>
```

3.7 JavaScript IF—ELSE Statements

- If else statement.

```
if(condition)
{
  code to be executed if condition is true
}
else
{
  code to be executed if condition is not true
}
```

- Example

```
var time = 10

if(time < 10)
{
  document.write("Good morning!");
}
else
{
  document.write("Good day!");
}
</script>
```

3.7 JavaScript IF—ELSE Statements

- If else if... else statement.

```
if(condition1)
{
  code to be executed if condition1 is true
}
else if (condition2)
{
  code to be executed if condition2 is true
}
else
{
  code to be executed if condition1 and condition2 are not true
}
```

3.7 JavaScript IF—ELSE Statements

- If else if... else statement.
- E.g:

```
var time= 10
if(time<10)
{
    document.write("<b>Good morning</b>");
}
else if (time>10 && time<16)
{
    document.write("<b>Good day</b>");
}
else
{
    document.write("<b>Hello World!</b>");
}
</script>
```

3.7 JavaScript IF—ELSE Statements

- Switch case statement.
 - E.g:

```
switch(n)
{
case 1:
    execute code block 1
    break;
case 2:
    execute code block 2
    break;
default:
    code to be executed if n is different from case 1 and 2
}
```

3.7 JavaScript switch Statements

- Switch case statement.
 - E.g:

```
theDay= 6
switch (theDay)
{
case 5:
    document.write("Finally Friday");
    break;
case 6:
    document.write("Super Saturday");
    break;
case 0:
    document.write("Sleepy Sunday");
    break;
default:
    document.write("I'm looking forward to this weekend!");
}
</script>
```

3.8 JavaScript Popup Boxes

- JavaScript has three kind of popup boxes:
 - Alert Box
 - Confirm Box
 - Prompt Box

1. Alert Box:

- ✓ It is often used if you want to make sure information comes from the user.
- ✓ When an alert box pops up, the user will have to click “OK” to proceed.
- ✓ Syntax

```
alert("sometext");
```

3.8 JavaScript Popup Boxes (...)

- Example:

```
<html>
<head>
<script type="text/javascript">
function show_alert()
{
alert("I am an alert box!");
}
</script>
</head>
<body>

<input type="button" onclick="show_alert()" value="Show alert box" />

</body>
</html>
```

3.8 JavaScript Popup Boxes (...)

2. Confirm Box:

- ✓ It is often used if you want the user to verify or accept something.
- ✓ When a confirm box pops up, the user will have to click either “OK” or “CANCEL” to proceed
- ✓ If the user clicks OK, the box returns true, else, It returns False
- ✓ Syntax:

```
confirm("sometext");
```

3.8 JavaScript Popup Boxes (...)

✓ Example:

```
<html>
<head>
<script type="text/javascript">
function show_confirm()
{
var r=confirm("Press a button");
if (r==true)

{
document.write("You pressed OK!");
}
else
{
document.write("You pressed Cancel!");
}
}
</script>
</head>
<body>

<input type="button" onclick="show_confirm()" value="Show confirm box" />

</body>
</html>
```

3.8 JavaScript Popup Boxes (...)

3. Prompt Box:

- ✓ It is often used if you want the user to input a value before entering a page.
- ✓ When a prompt box pops up, the user will have to click either "OK" or "Cancel" to proceed after entering an input value.
- ✓ If the user clicks "OK" the box returns the input value. If the user clicks "Cancel" the box returns null.
- ✓ Syntax

```
prompt("sometext","defaultvalue");
```

3.8 JavaScript Popup Boxes (...)

✓ Example

```
<html>
<head>
<script type="text/javascript">
function show_prompt()
{
var name=prompt("Please enter your name","Harry Potter");
if (name!=null && name!="")
{
document.write("Hello " + name + "! How are you today?");
}
}
</script>
</head>
<body>

<input type="button" onclick="show_prompt()" value="Show prompt box" />

</body>
</html>
```

3.9 JavaScript Functions

- ✓ A function will be executed by **an event or by a call** to the function.
- ✓ A function contains code that will be executed by an event or by a call to the function.
- ✓ You may **call a function from anywhere within a page** (or even from other pages if the function is embedded in an **external .js file**).
- ✓ **Function definition**

```
function functionname(var1,var2,...,varX)  
{  
  some code  
}
```

3.9 JavaScript Functions

✓ Function Example

```
<html>
<head>
<script type="text/javascript">
function displaymessage()
{
alert("Hello World!");
}
</script>
</head>

<body>
<form>
<input type="button" value="Click me!" onclick="displaymessage()" />
</form>
</body>
</html>
```

3.9 JavaScript Functions

✓ Function with return type Example

```
<html>
<head>
<script type="text/javascript">
function product(a,b)
{
return a*b;
}
</script>
</head>
```

```
<body>
<script type="text/javascript">
  document.write(product(4,3));
</script>

</body>
</html>
```

3.10 JavaScript LOOP

- Loops execute a block of code a specified number of times, or while a specified condition is true.
 - In JavaScript, there are two different kind of looks
 - a. **for** - loops through a block of code a specified number of times
 - b. **while** - loops through a block of code while a specified condition is true
-

3.10 JavaScript FOR LOOP

a. The for Loop

- The for loop is used when you know in advance how many times the script should run
- Syntax

```
for (var=startvalue;var<=endvalue;var=var+increment)  
{  
  code to be executed  
}
```

- Example

```
<html>  
<body>  
  <script type="text/javascript">  
    var i=0;  
    for (i=0;i<=5;i++)  
    {  
      document.write("The number is " + i);  
      document.write("<br />");  
    }  
  </script>  
</body>  
</html>
```

3.10 JavaScript FOR LOOP

b. The While Loop

- The while loop loops through a block of code while a specified condition is true.
- Syntax

```
while (var <= endvalue)
{
    code to be executed
}
```

```
<html>
<body>
<script type="text/javascript">
var i=0;
while (i<=5)
{
    document.write("The number is " + i);
    document.write("<br />");
    i++;
}
</script>
</body>
</html>
```

3.10 JavaScript FOR LOOP

c. The do...while Loop

- The do...while loop is a variant of the while loop. This loop will execute the block of code ONCE, and then it will repeat the loop as long as the specified condition is true.
- Syntax

```
do
{
  code to be executed
}
while (var<=endvalue);
```

3.10 JavaScript FOR LOOP

c. The do....while Loop

➤ Example

```
<html>
<body>
<script type="text/javascript">
var i=0;
do
{
  document.write("The number is " + i);
  document.write("<br />");

  i++;
}
while (i<=5);
</script>
</body>
</html>
```

3.10 For In Statement

- The `for...in` statement loops through the elements of an array or through the properties of an object.
- Syntax

```
for (variable in object)  
{  
  code to be executed  
}
```

- The code in the body of the `for...in` loop is executed once for each element/property.
-

3.10 For In Statement

– Example

```
<html>
<body>

<script type="text/javascript">
var x;
var mycars = new Array();
mycars[0] = "Saab";
mycars[1] = "Volvo";
mycars[2] = "BMW";

for (x in mycars)
{
  document.write(mycars[x] + "<br />");
}
</script>

</body>
</html>
```

3.11 JavaScript Break and Continue Statement

a. The break Statement

- The break statement will break the loop and continue executing the code that follows after the loop (if any).

– Eg:

```
<html>
<body>
<script type="text/javascript">
var i=0;
for (i=0;i<=10;i++)
{
  if (i==3)
  {
    break;
  }
  document.write("The number is " + i);
  document.write("<br />");
}
</script>
</body>
</html>
```

3.11 JavaScript Break and Continue Statement

b. The continue Statement

- The continue statement will break the current loop and continue with the next value.

– Eg:

```
<html>
<body>
<script type="text/javascript">
var i=0
for (i=0;i<=10;i++)
{
  if (i==3)
  {
    continue;
  }
  document.write("The number is " + i);
  document.write("<br />");
}
</script>
</body>
</html>
```

3.12 JavaScript Events

- An event is an action that occurs as per the user's instruction as input and gives the output in response.
 - We consider different types of inputs, such as mouse clicks, button presses, and when users press tab and text box change.
 - JavaScript implements a component called an event handler that helps to acknowledge certain actions to the events.
 - An **event handler** or **even listener** is a section of code that can be considered a user-defined JavaScript function executed when a particular event fires.
-

Types of JavaScript's

1. **HTML5**: This defines the multiple types of events, particularly using HTML script, and the numerous categories like “**Submit** and **Input types of Button**”, and events such as “**beforeload**” and “**hashchange**”.
 2. **Browser Object Models (BOM)**: This category of JavaScript comes from W3 specs. Touchscreen devices using “**touchstart**”, “**touchend**” events are the perfect examples of BOM.
 3. **W3 DOM**: It performs different types of DOM events along with the form elements
-

3.12 JavaScript Events

- Examples of events:
 - A mouse click
 - A web page or an image loading
 - Mousing over a hot spot on the web page
 - Selecting an input field in an HTML form
 - Submitting an HTML form
 - A keystroke
-

Common Event Listeners

JavaScript listeners' definitions have specific types of methods:

1. **Onload**: When your page loads, it performs accordingly.
 2. **Onclick**: When a user clicks on a button or inputs it occurs.
 3. **Onmouseover**: When a user mouses over on the button.
 4. **Onfocus**: Certain scenarios when a user keeps the cursor in a form field
 5. **Onblur**: If a particular form field leaves within it.
-

3.12 JavaScript Events

- **onLoad and OnUnload**
 - Those events are triggered when the user enters or leaves the page.
 - Both the onLoad and onUnload events are also often used to deal with cookies that should be set when a user enters or leaves a page.

3.12 JavaScript Events

- **onFocus, onBlur and onChange**

- The onFocus, onBlur and onChange events are often used in combination with validation of form fields.
- Example

```
<input type="text" size="30" id="email" onchange="checkEmail()">
```

- **OnSubmit**

- The onSubmit event is used to validate ALL form fields before submitting it.
-

3.12 JavaScript Events

- **onMouseOver and onMouseOut**

- onMouseOver and onMouseOut are often used to create "animated" buttons.
- Below is an example of an onMouseOver event.
- An alert box appears when an onMouseOver event is detected:

```
<a href="http://www.w3schools.com" onmouseover="alert('An onMouseOver  
event');return false"></a>
```

3.12 JavaScript Events - Tutorial

```
<body onload="message()">
  Enter your Fname:
  <input type="text" id="Firstname" onkeyup="myKeyUpFunction()">
  <br/>
  <br/>
  Enter your Email:
  <input type="text" id="Email"/>
  <br/>
  <p>My First name is: <span style="color:green;font-size:40px;"id="Test"></span></p>
  <h2 id="demo">Test Mouse over me</h2>
</body>
```

```
//Onload
function message(){
  alert("I am the message")
}
//OnKeyUp
function myKeyUpFunction() {
  var input = document.getElementById("Firstname").value;
  document.getElementById("Test").innerHTML = input;
}
//OnChange
document.getElementById("Email").onchange = function() {
  myFunctionchange()
};
function myFunctionchange() {
  var x = document.getElementById("Email");
  x.value = x.value.toLowerCase();
}
// onFocus
document.getElementById("Firstname").addEventListener("focus", myFunction);
function myFunction() {
  document.getElementById("Firstname").style.backgroundColor = "Red";
}
```

```
// onBlur
document.getElementById("Email").addEventListener("blur", myFunctionBlur);
function myFunctionBlur() {
  alert("your Input element lost focus.");
}
//<!--Mouse over-->
document.getElementById("demo").onmouseover = function() {
  mouseOver()
};
function mouseOver() {
  document.getElementById("demo").style.color = "Red";
}
//<!--Mouse Out-->
document.getElementById("demo").onmouseout = function() {
  mouseOut()
};
function mouseOut() {
  document.getElementById("demo").style.color = "Cyan";
}
```

3.12 JavaScript Events – HTML with Functions Call

```
<body onload="message()">
  Enter your Fname:
  <input type="text" id="Firstname" onkeyup="myKeyUpFunction()">
  <br/>
  <br/>
  Enter your Email:
  <input type="text" id="Email"/>
  <br/>
  <p>My First name is: <span style="color: green;font-size:40px;"id="Test"></span></p>
  <h2 id="demo">Test Mouse over me</h2>
</body>
```

3.12 JavaScript Events – Functions Definition

```
//Onload
function message(){
    alert("I am the message")
}

//OnKeyUp
function myKeyUpFunction() {
    var input = document.getElementById("Firstname").value;
    document.getElementById("Test").innerHTML = input;
}

//OnChange
document.getElementById("Email").onchange = function() {
    myFunctionchange()
};

function myFunctionchange() {
    var x = document.getElementById("Email");
    x.value = x.value.toLowerCase();
}

// onfocus
document.getElementById("Firstname").addEventListener("focus", myFunction);
function myFunction() {
    document.getElementById("Firstname").style.backgroundColor = "Red";
}
```

3.12 JavaScript Events – Function Definitions

```
// onBlur
document.getElementById("Email").addEventListener("blur", myFunctionBlur);
function myFunctionBlur() {
    alert("your Input element lost focus.");
}
//<!--Mouse over-->
document.getElementById("demo").onmouseover = function() {
    mouseOver()
};
function mouseOver() {
    document.getElementById("demo").style.color = "Red";
}
//<!--Mouse Out-->
document.getElementById("demo").onmouseout = function() {
    mouseOut()
};
function mouseOut() {
    document.getElementById("demo").style.color = "Cyan";
}
```

3.13 JavaScript Objects Introduction

- **JavaScript is an Object Oriented Programming (OOP) language**
 - An OOP language allows you to define your own objects and make your own variable types.
 - There are **Defined JavaScript Object** and **Built-In JavaScript Object**
 - Note that, an object is a special kind of data. It has properties and methods.
 - Properties are the values associated with an object

```
<script type="text/javascript">  
var txt="Hello World!";  
document.write(txt.length);  
</script>
```

- Methods are the actions that can be performed on objects.

```
<script type="text/javascript">  
var str="Hello world!";  
document.write(str.toUpperCase());  
</script>
```

3.13 JavaScript Date Object

- The Date object is used to work with dates and times.
- The following code create a Date object called myDate.

```
var myDate=new Date()
```

- The Date object will automatically hold the current date and time as its initial value..
- Compare Two Dates

```
var myDate=new Date();  
myDate.setFullYear(2010,0,14);  
var today= new Date();  
  
if(myDate>today)  
{  
    alert("Today is before 14th January 2010");  
}  
else  
{  
    alert("Today is after 14th January 2010");  
}
```

3.13 JavaScript Date Object

- Example

```
<script type="text/javascript">
//Write a "Good morning" greeting if
//the time is less than 10

var d=new Date();
var time=d.getHours();

if (time<10)
{
  document.write("<b>Good morning</b>");
}
</script>
```

3.13 JavaScript Date Object

- Example

```
<script type="text/javascript">
//If the time is less than 10, you will get a "Good morning" greeting.
//Otherwise you will get a "Good day" greeting.

var d = new Date();
var time = d.getHours();

if (time < 10)
{
    document.write("Good morning!");
}
else
{
    document.write("Good day!");
}
</script>
```

3.14 JavaScript Date Object

- E.g:

```
<script type="text/javascript">
var d = new Date()
var time = d.getHours()
if (time < 10)
{
    document.write("<b>Good morning</b>");
}
else if (time > 10 && time < 16)
{
    document.write("<b>Good day</b>");
}
else
{
    document.write("<b>Hello World!</b>");
}
</script>
```

3.14 JavaScript Date Object

— E.g:

```
<script type="text/javascript">
//You will receive a different greeting based
//on what day it is. Note that Sunday=0,
//Monday=1, Tuesday=2, etc.

var d=new Date();
theDay=d.getDay();
switch (theDay)
{
case 5:
    document.write("Finally Friday");
    break;
case 6:
    document.write("Super Saturday");
    break;
case 0:
    document.write("Sleepy Sunday");
    break;
default:
    document.write("I'm looking forward to this weekend!");
}
</script>
```

3.15 JavaScript Array Object

- **The Array Object is used to store multiple values in a single variable**

- The array is a special variable, which can hold more than one value, at a time.
- If you have a items (a list of car names, for example), storing the cars in single variables could look like this:

```
$cars1="Saab";  
$cars2="Volvo";  
$cars3="BMW";
```

- However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?
 - The best solution here is to use an array!
-

3.15 JavaScript Array Object

- **Create an Array**

- Creates an Array object called myCars: with values assignment

```
var myCars=new Array();  
myCars[0]="Saab";  
myCars[1]="Volvo";  
myCars[2]="BMW";
```

- When you want to control the size of an array, pass an integer argument

```
var myCars=new Array(3);  
myCars[0]="Saab";  
myCars[1]="Volvo";  
myCars[2]="BMW";
```

- Other way

```
var myCars=new Array("Saab","Volvo","BMW");
```

3.15 JavaScript Array Object

- **Access an Array**

- You can refer to a particular element in an array by referring to the name of the array and the index number. The index number starts at 0.

```
document.write(myCars[0]);
```

- **Modify in Array**

- To modify a value in an existing array, just add a new value to the array with a specified index number:

```
myCars[0]="Opel";
```

```
document.write(myCars[0]);
```

3.15 JavaScript Validation Form

- **JavaScript can be used to validate data in HTML forms before sending off the content to a server**
 - ✓ Form data that typically are checked by a JavaScript could be:
 - has the user left required fields empty?
 - has the user entered a valid e-mail address?
 - has the user entered a valid date?
 - has the user entered text in a numeric field?
-

3.15 JavaScript Validation Form

Required field

```
<html>
<head>
<script type="text/javascript">
function validate_required(field,alerttxt)
{
with (field)
{
if (value==null||value=="")
{
alert(alerttxt);return false;
}
else
{
return true;
}
}
}
}

function validate_form(thisform)
{
with (thisform)
{
if (validate_required(email,"Email must be filled out!")==false)
{email.focus();return false;}
}
}
</script>
</head>

<body>
<form action="submit.htm" onsubmit="return validate_form(this)" method="post">
Email: <input type="text" name="email" size="30">
<input type="submit" value="Submit">
</form>
</body>

</html>
```

3.15 JavaScript Validation Form

E-mail Validation

```
<html>
<head>
<script type="text/javascript">
function validate_email(field,alerttxt)
{
with (field)
{
apos=value.indexOf("@");
dotpos=value.lastIndexOf(".");
if (apos<1||dotpos-apos<2)
{alert(alerttxt);return false;}
else {return true;}
}
}
}
```

```
function validate_form(thisform)
{
with (thisform)
{
if (validate_email(email,"Not a valid e-mail address!")==false)
{email.focus();return false;}
}
}
</script>
</head>

<body>
<form action="submit.htm" onsubmit="return validate_form(this);" method="post">
Email: <input type="text" name="email" size="30">
<input type="submit" value="Submit">
</form>
</body>

</html>
```

3.16 JavaScript Animation

- It is possible to use JavaScript to create animated images.
- The trick is to let a JavaScript change between different images on different events.
- In the following example we will add an image that should act as a link button on a web page.
- We will then add an onMouseOver event and an onMouseOut event that will run two JavaScript functions that will change between the images.

```
<a href="http://www.w3schools.com" target="_blank">  
</a>
```

3.16 JavaScript Animation

- Example:

```
<html>
<head>
<script type="text/javascript">
function mouseOver()
{
document.getElementById("b1").src="b_blue.gif";
}
function mouseOut()
{
document.getElementById("b1").src="b_pink.gif";
}
</script>
</head>

<body>
<a href="http://www.w3schools.com" target="_blank">
</a>
</body>
</html>
```

3.18 JSON & AJAX

- Next:

Talking to the server without reloading the page



```
const data = await  
fetch(url);
```

```
const json = await  
data.json();
```

```
console.log(json);
```

What We'll Cover Today



03

JSON Fundamentals

Syntax, data types, and example data



01

AJAX Basics

What it is, why it matters, when to use it



04

JSON + AJAX

Parsing, using, and rendering JSON into HTML



02

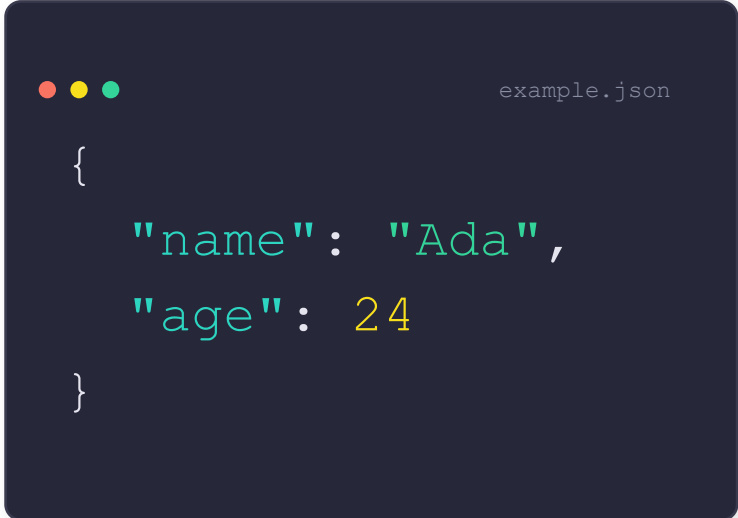
AJAX in Code

XMLHttpRequest and the modern Fetch API

JSON FUNDAMENTALS

JavaScript Object Notation

- A lightweight, text-based format for storing and exchanging data.
- Easy for humans to read and write, easy for machines to parse.
- Language-independent — used by JavaScript, Python, Java, PHP, and almost every API.
- It is the standard data format AJAX requests send and receive today.



```
{  
  "name": "Ada",  
  "age": 24  
}
```

JSON FUNDAMENTALS

JSON Syntax Rules & Data Types

- **Data** is written as key : value pairs
- **Keys** must always be double-quoted strings
- **Values** can be: string, number, boolean, null, array, or object
- Objects use { } · Arrays use []
- Commas separate items — no trailing comma allowed

```
types.json
{
  "text": "hello",
  "count": 3,
  "active": true,
  "note": null,
  "tags": ["js", "web"]
}
```

JSON FUNDAMENTALS

Example: A JSON Data Set

Real APIs usually return an array of objects — for example, a list of students:

```
students.json

[
  {
    "id": 1,
    "name": "Aline",
    "course": "Web Design"
  },
  {
    "id": 2,
    "name": "Eric",
    "course": "Web Design"
  }
]
```

Using JSON Inside JavaScript

JSON.parse()

Turns a JSON string into a real JS object/array

JSON.stringify()

Turns a JS object/array into a JSON string

```
parse.js

const jsonText =
  '{"name": "Ada"}';

const obj =
  JSON.parse(jsonText);
console.log(obj.name);
// Ada

const backToText =
  JSON.stringify(obj);
```

JSON + HTML

Rendering JSON Into Simple HTML

Once data is a JS object, write it straight into the page with the DOM:

```
index.html  
  
<div  
  id="profile"></div>
```

```
script.js  
  
const user = { name: "Ada"  
};  
  
document.getElementById("p  
rofile")  
  .innerHTML = `Hello,  
${user.name}!`;
```

BROWSER PREVIEW

Hello, Ada!

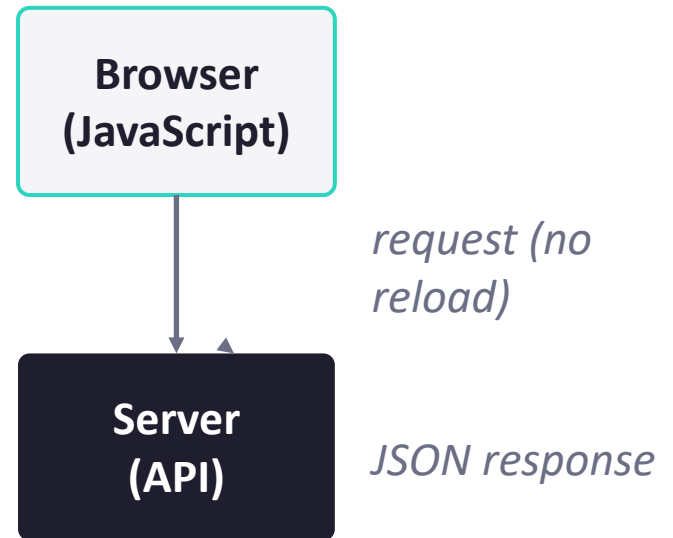
The div's content is replaced with data pulled from the JS object — this is exactly what AJAX responses let you do, but with live server data.

AJAX BASICS

What Is AJAX?

AJAX = **Asynchronous JavaScript And XML**

- ◆ A technique — not a language or library — for exchanging data with a server in the background.
- ◆ Let's update a web page part of itself without a full page reload.
- ◆ Despite the name, modern AJAX almost always uses JSON, not XML.
- ◆ Built on the browser's XMLHttpRequest object, or today, the Fetch API.



Why AJAX? The Problem It Solves



Without AJAX

- Every action reloads the entire page
- Browser re-downloads HTML, CSS, JS, and images
- Screen flashes white; scroll position resets
- Feels slow — bad experience on forms & search

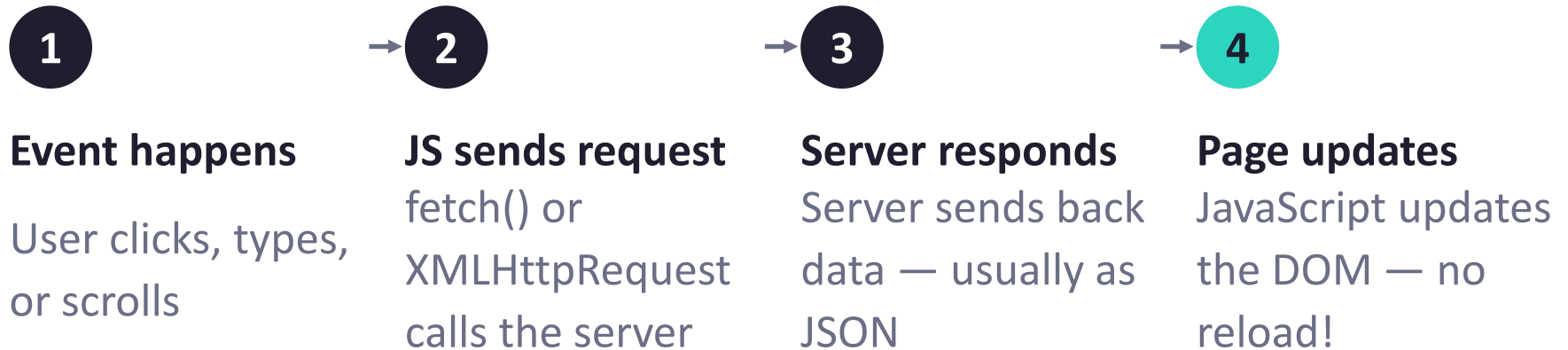


With AJAX

- Only the needed data is sent/received
- Page stays put — only part of it updates
- Faster, smoother, app-like experience
- Powers search suggestions, likes, live feeds

AJAX BASICS

How AJAX Works, Step by Step



Key idea: Step 2–3 happen “asynchronously” — the browser keeps working (animations, clicks) while it waits for the server's reply.

AJAX BASICS

When Should You Use AJAX?



Live search

Show suggestions as the user types



Infinite scroll

Load more posts/products on scroll



Form validation

Check a username/email without reloading



Live data

Refresh prices, chats, or notifications



Partial updates

Update a cart or like-count in place



Third-party APIs

Pull weather, maps, or currency data

Classic AJAX: XMLHttpRequest

The original way browsers make AJAX calls. Verbose, but still good to recognize:

```
const xhr = new XMLHttpRequest();  
xhr.open("GET", "data.json");  
  
xhr.onload = function () {  
    if (xhr.status === 200) {  
        console.log(xhr.responseText);  
    }  
};  
  
xhr.send();
```

script.js

Modern AJAX: The Fetch API

Cleaner and Promise-based — the standard choice in modern JavaScript:

```
fetch("data.json")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// or, with async/await:
async function loadData() {
  const response = await fetch("data.json");
  const data = await response.json();
  console.log(data);
}
```

JSON + AJAX

index.html

```
<ul id="list"></ul>
```

script.js

```
fetch("students.json")
  .then(res =>
res.json())
  .then(students => {
    const list =
document.getElementById
("list");
    students.forEach(s
=> {
      list.innerHTML +=
`<li>${s.name}</li>`;
    });
  });
```

BROWSER PREVIEW

- Aline
- Eric

The list starts empty in the HTML. Once `fetch()` resolves, JavaScript loops through the JSON array and injects one `<li>` per student — live, without reloading the page.

AJAX - API

```
<!DOCTYPE html>
<html>
<head>
    <title>Fetch API Example</title>
</head>

<body>

<p id="p1"></p>
<p id="p1warn" style="color:red;"></p>
```

```
async function loadData() {
    try {
        const response = await fetch("https://dummyjson.com/products");

        const data = await response.json();

        console.log(data);

        document.getElementById("p1").innerHTML =
            "Total Products: " + data.total;
    }
    catch(error) {
        document.getElementById("p1warn").innerHTML = error.message;
    }
}
loadData();
```

API (Public)

API	Base URL	Example Endpoint	Purpose
JSONPlaceholder	<code>https://jsonplaceholder.typicode.com</code>	<code>/posts</code> , <code>/users</code> , <code>/comments</code>	Fake REST API for CRUD testing  GitHub +1
DummyJSON	<code>https://dummyjson.com</code>	<code>/products</code> , <code>/users</code> , <code>/posts</code>	Products, users, authentication
ReqRes	<code>https://reqres.in/api</code>	<code>/users?page=2</code>	User authentication and CRUD practice
Random User	<code>https://randomuser.me/api</code> <code>/</code>	<code>/</code>	Generate random users
REST Countries	<code>https://restcountries.com</code> <code>/v3.1</code>	<code>/all</code> , <code>/name/rwanda</code>	Country information

PRACTICE & DEBUGGING

Common Errors & How to Handle Them

**CORS error**

Server must allow requests from your page's origin

**404 / 500 status**

Always check `response.ok` before using the data

**Invalid JSON**

A trailing comma or missing quote breaks `JSON.parse()`

**No error handling**

Wrap calls in `try/catch` or add `.catch()`

```
try {  
  const res = await fetch(url);  
  if (!res.ok) throw new Error(res.status);  
  const data = await res.json();  
} catch (err) {  
  console.error("Request failed:", err);  
}
```

RECAP

-  AJAX updates a page without a full reload, using background requests.
 -  `fetch()` is the modern way to make AJAX calls in JavaScript.
 -  JSON is the standard text format for exchanging structured data.
 -  `JSON.parse()` turns text into objects; `JSON.stringify()` reverses it.
 -  Always handle errors — networks and servers can fail.
-

Practice Exercise



Build a mini profile-card page:

- ◆ Create a JSON file with 3 fields: name, role, and skills (an array).
 - ◆ Use `fetch()` to load the JSON file when the page opens.
 - ◆ Use `JSON.parse()` / `response.json()` to turn it into a JS object.
 - ◆ Write the data into an empty `<div>` using the DOM — no page reload.
-

End
